

Elevadores Casa Roja

Simulador didático de despacho de elevadores com lógica fuzzy — versão web

Disciplina: Inteligência Computacional Aplicada (ICA) — PPGCC/IFCE

Aluno(a): Luiz Henrique dos Anjos Santos

Professor: Cláudio Sá

Demo ao vivo: <https://henriqueanjos12.github.io/elevadores-fuzzy-web/>

Repositório: <https://github.com/Henriqueanjos12/elevadores-fuzzy-web>

1. Objetivo

Este trabalho implementa um sistema de gerenciamento de atendimento a chamadas para um conjunto de três elevadores usando lógica fuzzy (Mamdani), conforme o exercício "Sistema de gerenciamento de atendimento a chamadas para um conjunto de três elevadores" da apostila da disciplina. A decisão de qual elevador atende cada chamada é tomada por um controlador fuzzy que combina a distância até o andar da chamada e a lotação atual de cada elevador, produzindo uma prioridade de atendimento.

Esta é a **versão web** do projeto — HTML, CSS e JavaScript puro, sem dependências nem etapa de build, publicada como site estático no GitHub Pages para ser aberta direto no navegador (inclusive pelo celular) e usada ao vivo em sala de aula. Existe também uma versão desktop equivalente em Python/CustomTkinter, no repositório irmão, com a mesma lógica de decisão.

2. Especificação do problema

O cenário simulado é o **Edifício Casa Roja**: 3 elevadores e, por padrão, 12 pavimentos (Térreo a 11º andar) — quantidade de pavimentos e capacidade máxima por elevador são configuráveis em tempo real na própria interface. Cada pavimento tem botões de chamada (▲ subir / ▼ descer, respeitando as direções fisicamente possíveis em cada extremo do prédio). O fluxo de atendimento é modelado em **duas fases**, como um elevador real: apertar o botão externo só registra andar e direção — só quando o elevador designado chega com a porta aberta é que o "painel interno" pergunta quantas pessoas embarcam e para quais andares.

3. A lógica por trás da regra de lotação

A base de regras favorece elevadores com lotação mais baixa — quanto menos carregada a cabine, maior a prioridade atribuída pelo controlador. Uma forma de dar sentido físico a essa regra (e não apenas tratá-la como uma preferência arbitrária) é pensar no sistema de **contrapeso** dos elevadores reais: um contrapeso, ligado à cabine pelo mesmo cabo sobre a polia, é dimensionado para equilibrar a cabine vazia mais aproximadamente **metade** da carga máxima. Dependendo de quanto a cabine está carregada e para qual lado ela precisa se mover, a gravidade tende a ajudar o motor ou a

atrapalhar — por exemplo, uma cabine mais leve que o contrapeso subindo é ajudada pelo contrapeso descendo, e no caso extremo de um elevador **parado no último andar, totalmente lotado**, bastaria "soltar" a cabine para ela descer sozinha até o térreo, com o motor apenas freando.

Essa é apenas uma **explicação intuitiva** para o comportamento da lógica fuzzy adotada aqui — o projeto não calcula nem otimiza consumo de energia; é só uma forma de justificar, com uma analogia física concreta, por que "lotação baixa → prioridade alta" é uma regra sensata, e não meramente uma convenção. Uma variante desse raciocínio, que também leva em conta o **sentido** do deslocamento (subir vs. descer), foi explorada separadamente à parte deste projeto (ver Seção 9).

4. Fundamentação teórica

O controlador segue o modelo de inferência de **Mamdani**, com as etapas clássicas de fuzzificação, processamento das regras e defuzzificação:

- **Funções de pertinência trapezoidais** $[a, b, c, d]$: sobem de 0 (em a) a 1 (em b), permanecem em 1 até c , e descem a 0 em d — um triângulo é o caso particular $b = c$.
- **Operador E (antecedente composto)**: interseção pelo mínimo, $\mu(A \wedge B) = \min[\mu_A(x), \mu_B(y)]$.
- **Implicação de Mamdani**: a saída de cada regra é o mínimo entre a força de ativação da regra e a função de pertinência do termo de saída, $\mu_{C'}(w) = \min[\alpha, \mu_C(w)]$.
- **Agregação**: as saídas de todas as regras que apontam para o mesmo termo de saída são combinadas pelo máximo, $\mu_R(w) = \max[\mu_{C'1}(w), \mu_{C'2}(w), \dots]$.
- **Defuzzificação por centroide**, sobre um universo de saída discreto (0 a 100, passo 1): $\text{saída} = \frac{\sum(w \cdot \mu_R(w))}{\sum(\mu_R(w))}$.

Essas definições seguem diretamente o conteúdo da disciplina (Cap. 10 — Noções de Controle Nebuloso; e a seção "Sistema Nebuloso – Inferência Mamdani" da apostila complementar), inclusive o exemplo numérico de avaliação de crédito usado para introduzir a implicação de Mamdani.

5. Modelagem do controlador fuzzy

O controlador usa duas variáveis de entrada e uma de saída:

Variável	Universo	Termos linguísticos
distancia	0 a N-1 pavimentos (N = andares do prédio)	próxima, média, distante
lotacao	0 a 100 %	baixa, média, alta
prioridade (saída)	0 a 100	muito baixa, baixa, média, alta, muito alta

Os pontos das pertinências padrão são igualmente espaçados no universo de cada variável (passo = universo / 4), o que garante simetria e evita "ilhas" (nenhum ponto do universo fica com todos os termos zerados ao mesmo tempo). O usuário pode arrastar os vértices dos trapézios na própria interface para explorar outras formas em tempo real, sem precisar reiniciar a simulação.

A base de regras cobre a **grade completa** de `distancia` (3 termos) × `lotacao` (3 termos) — 9 regras, sem sobra nem lacuna, do tipo:

SE `distancia` é **próxima** E `lotacao` é **baixa** ENTÃO `prioridade` é **muito alta**

Antes de qualquer chamada entrar na máquina fuzzy, um **filtro determinístico** descarta (prioridade 0 direto) elevadores que fisicamente não fazem sentido para a chamada: lotados, ou em movimento e já passados do andar da chamada. A interface mostra o motivo exato do descarte para cada elevador eliminado dessa forma.

6. Arquitetura da solução (versão web)

Implementada em módulos ES (`import / export`), sem build step nem dependências externas — basta servir os arquivos por HTTP. A estrutura espelha diretamente os módulos da versão Python original:

Arquivo	Responsabilidade
<code>js/fuzzy.js</code>	controlador fuzzy: pertinências, regras, inferência Mamdani + centroide
<code>js/models.js</code>	elevador/chamada/passageiro como objetos simples
<code>js/despachante.js</code>	escolhe o elevador (fuzzy ou mais próximo), com cascata de desempate
<code>js/simulador.js</code>	laço de ciclos discretos, embarque/desembarque, eventos
<code>js/building.js</code>	<canvas> do prédio: poços, portas, botões de chamada
<code>js/charts.js</code>	gráficos de pertinência com edição ao vivo (arrastar vértices)
<code>js/main.js</code>	orquestração geral e laço de simulação

7. Funcionalidades

- **Prédio configurável:** número de andares e capacidade máxima por elevador ajustáveis em tempo real.
- **Fluxo em duas fases** de embarque, como um elevador real.
- **Gráficos de pertinência editáveis** por arraste, com o motor de inferência reagindo em tempo real.
- **Tabela de decisão fuzzy:** compara os 3 elevadores lado a lado para a última chamada, com o escolhido destacado e o motivo de descarte dos demais quando aplicável.
- **Calculadora de aptidão:** configura cada elevador manualmente (posição, carga, condição) e calcula a prioridade para uma chamada hipotética, sem afetar a simulação em andamento.
- **Log de eventos** e **dois algoritmos de despacho** (fuzzy vs. mais próximo) para comparar.

8. Capturas de tela

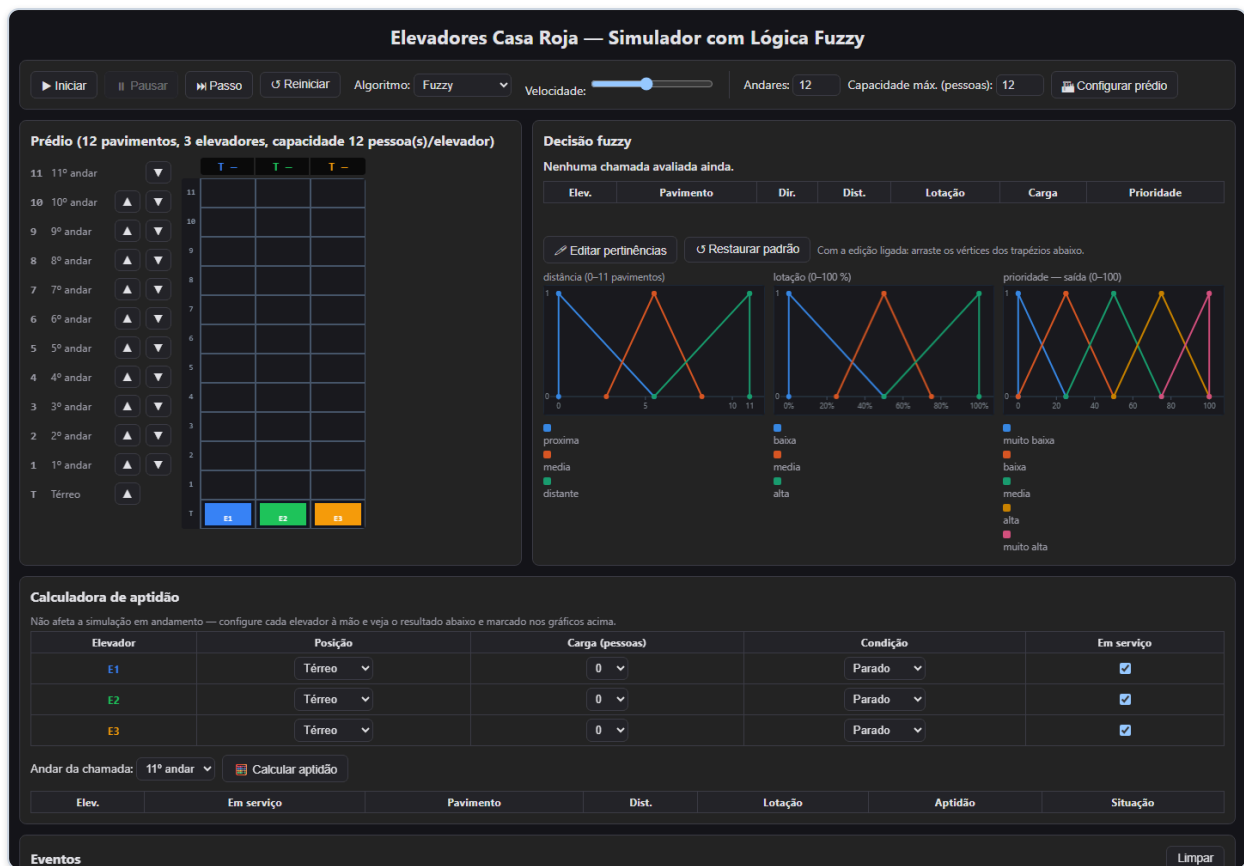


Figura 1 — Estado inicial: prédio, decisão fuzzy e calculadora de aptidão.

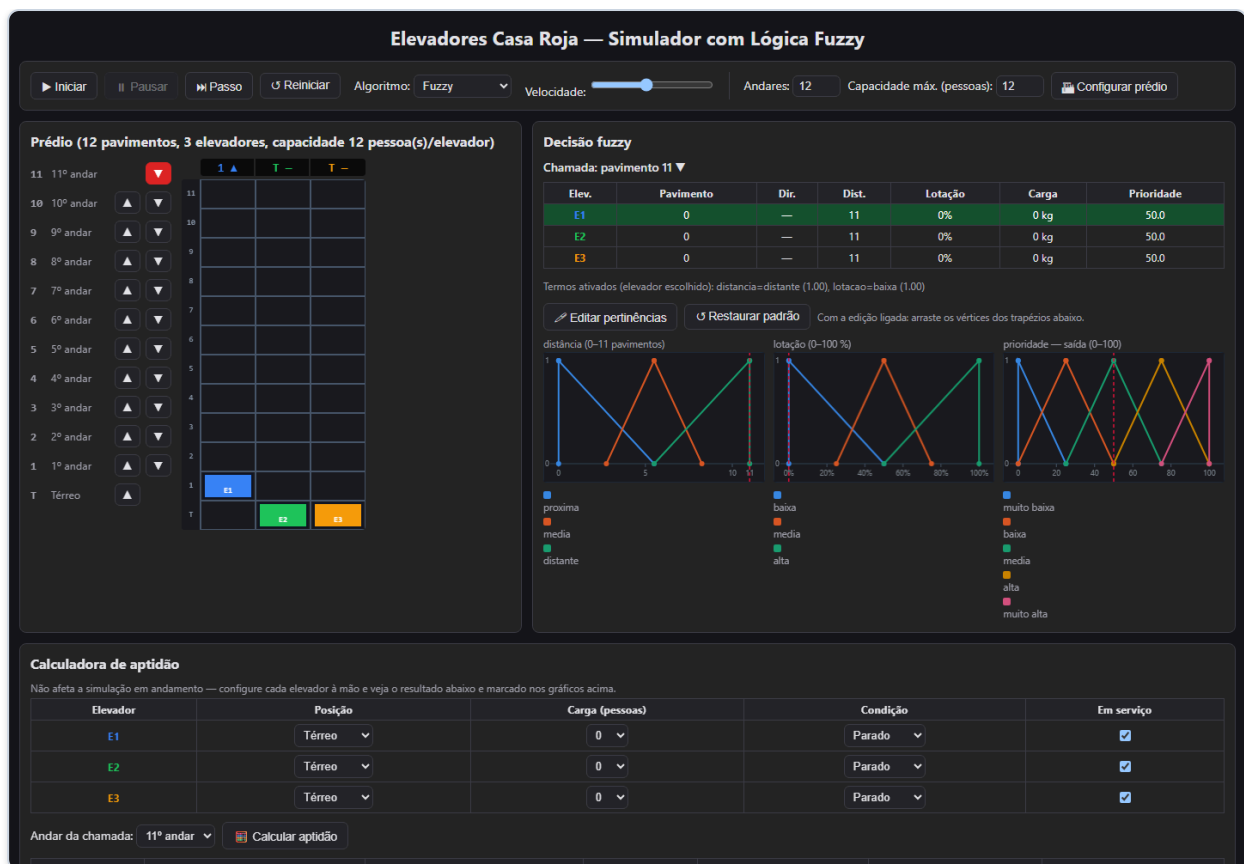


Figura 2 — Chamada ativa: o Elevador 1 é escolhido (linha verde), com a entrada marcada nos três gráficos de pertinência.



Figura 3 — Modo de edição: arrastar os vértices dos trapézios reconstrói o controlador fuzzy imediatamente.

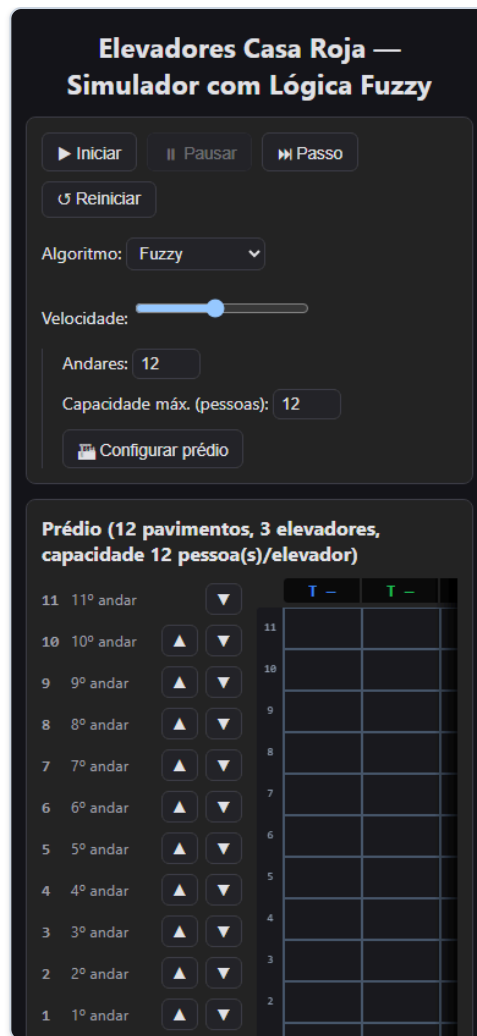


Figura 4 — Layout responsivo em largura de celular (390px).

9. Simplificações assumidas

Para manter o foco didático, algumas partes de uma especificação mais ampla foram deliberadamente deixadas de fora desta versão:

- A entrada fuzzy considera **distância** e **lotação atual**; "estimativa de demanda por andar" e "previsão de paradas" chegaram a ser exploradas (esta última como uma terceira variável fuzzy) mas foram removidas para manter a base de regras simples o bastante para ser explicada ao vivo em aula.
- A lotação usada hoje não distingue o **sentido** do deslocamento — trata "lotação baixa" como sempre favorável, quando a intuição do contrapeso (Seção 3) só vale assim para subidas (para descidas, seria o oposto: cabine mais cheia). Uma variante que também considera o sentido (subir/descer) foi prototipada separadamente, à parte deste projeto.
- A máquina de inferência implementada é a de **Mamdani**; um motor Takagi-Sugeno (TS) não foi implementado nesta versão.
- Geração automática/aleatória de chamadas e o painel de métricas detalhadas existem na versão desktop, mas não nesta versão web — aqui as chamadas são sempre criadas manualmente pelos botões do prédio.

10. Como acessar

- **Demo ao vivo (não precisa instalar nada):**
<https://henriqueanjos12.github.io/elevadores-fuzzy-web/>
- **Código-fonte:**
<https://github.com/Henriqueanjos12/elevadores-fuzzy-web>
- **Rodando localmente:** `python -m http.server 8000` na pasta do projeto, depois abrir `http://localhost:8000/` (precisa de um servidor HTTP porque os módulos JavaScript não carregam via `file://`).

11. Conclusão

A versão web reproduz fielmente o motor de decisão fuzzy da versão Python original (mesmas pertinências, mesmas 9 regras, mesmo filtro determinístico), num formato que roda em qualquer navegador sem instalação — o que a torna adequada tanto para demonstração em sala de aula quanto para exploração individual pelos alunos depois. A edição ao vivo das funções de pertinência e a calculadora de aptidão tornam concretos, de forma interativa, os conceitos de fuzzificação, inferência Mamdani e defuzzificação apresentados na disciplina.